

Домашнее задание 9

Лиза Фоменко

Задание 1

Вам нужно выбраться из лабиринта. Вы не знаете, сколько в нем комнат, и какая у него карта. По всем коридорам можно свободно перемещаться в обе стороны, все комнаты и коридоры выглядят одинаково (комнаты могут отличаться только количеством коридоров). Пусть m - суммарное количество коридоров между комнатами. Предложите алгоритм, который находит выход из лабиринта или доказывает, что его нет, за $O(m)$ переходов между комнатами. В вашем расположении имеется неограниченное количество монет, которые вы можете оставлять в комнатах. Минотавр мертв, так что в лабиринте больше никого.

Мне кажется эту задачу можно легко решить, представив, что комнаты = вершины, а коридоры = ребра некоторого графа $G(V, E)$, где $|E| = m$. Тогда, предположим, что мы стоим в какой-то комнате из $v \in V$. Начнем обход графа-лабиринта в глубину, при этом мы бы хотели пометить пару комната-коридор. Так как у нас монеток не ограничено, то ничего не запрещает нам класть монетку в комнате около каждого входа в коридор, в который мы идем. Тогда алгоритм такой:

```
first_v = v          # вершина, в которой мы находимся

while True:
    if curr_v == 'EXIT':          # если нашли выход
        return 'found exit!'

    elif в вершине-комнате curr_v есть непосещенные коридоры:      # если есть возможность продолжать DFS
        выбираем коридор
        кладем монетку около его входа
        идем по коридору в вершину-комнату new_v
        prev_v = curr_v
        curr_v = new_v

    else:                        # нашли комнату, где прошли все коридоры, возвращаемся обратно
        curr_v = prev_v

if curr_v == first_v:          # если вернулись туда, откуда начинали, значит, выхода нет
    return 'no exit found'
```

Получается, что мы реализовали DFS, при этом каждую комнату мы посетили не более $2k$ раз, где k = количество коридоров, ведущих из этой комнаты, а каждое ребро - не более 2 раз (1 раз в одном направлении, 2 раз - в обратном). Как бы единственное отличие задачи от DFS в том, что мы используем монетки для мечения пройденных коридоров из данной комнаты. Коректность DFS мы вроде доказывали, и так как каждое ребро мы смотрим не более 2 раз, то сложность $O(m)$ (на самом то деле $O(m + n)$ ну в алгоритмическом смысле $O(m + n)$, где n = количество вершин-комнат. Но условно как бы ок, $O(m)$, потому что нам нужна асимптотика от переходов между комнатами, поэтому $O(m)$ works)))

Задание 2

Предложите алгоритм, выясняющий, есть ли в неориентированном графе циклы нечетной длины. Оцените асимптотику его работы. Докажите его корректность.

Мы на лекции доказали, что любой ациклический граф двудольный.

Докажем, что графы, имеющие циклы нечетной длины, не являются двудольными, а графы только с четными циклами двудольными являются. Тогда нам останется только проверить графы на двудольность

Докажем с помощью раскрашивания вершин.

Будем использовать обход в глубину. Давайте возьмем произвольную вершину v_0 и поставим ей метку 0. Для всех вершин на расстоянии 1 от нее ставим метку 1, для вершин на расстоянии 2 - метку 0 и так далее: для вершин на четном расстоянии ставим метку 0, для вершин на нечетном - 1.

Пусть теперь в нашем графе есть циклы, то есть существует путь из v_i в v_i , проходящий через набор вершин $v_{i+1}, \dots, v_{i+k}$. Пусть v_i имеет метку 0. Для этой вершины так же характерно, что все вершины на четном расстоянии от нее окрашены так же, как она, то есть имеют метку 0, остальные - метку 1. Тогда, чтобы граф оставался двудольным, после прохождения по циклу, следуя этому правилу, натолкнувшись снова на вершину v_i , исходя из нее, мы должны вновь поставить ей метку 0, то есть "вершина v_i находится на четном расстоянии от себя самой" то есть длина ее цикла четна. Если же цикл будет нечетным, то присуждая четным и нечетным вершинам разные метки, мы будем вынуждены поставить v_i , имеющей метку 0, новую метку 1, то есть нашли противоречие, и наш граф двудольным не является.

Доказали: если в графе есть цикл нечетной длины, значит, он не двудольный.

Дальше докажем обратное: если граф двудольный, то в нем нет циклов нечетной длины. Докажем. Пусть существует двудольный граф, который мы можем разделить на доли А, В, а в нем есть цикл нечетной длины из вершины v_i . Пусть v_i принадлежит А. Тогда в В содержатся все вершины, находящиеся на нечетном расстоянии от v_i , а в А - на четном расстоянии. Мы знаем, что существует цикл нечетной длины $v_i \rightarrow \dots \rightarrow v_i$. Значит, существует путь нечетной длины от v_i в v_i , значит, v_i находится на нечетном расстоянии от самой себя и должна входить в долю В тоже. Но это невозможно по условию, так как в двудольном графе каждая вершина принадлежит только одной доле. Значит, предположение неверно.

Тогда мы знаем: любой ациклический граф двудольный (лекция); граф с циклами четной длины - двудольный (выше); граф с циклом нечетной длины - НЕ двудольный. Тогда проверка проста: проверям, является ли наш неорграф двудольным. Если нет, то это значит, что в нем есть цикл нечетной длины. Такую проверку можно делать путем окраски графа в 2 цвета с помощью DFS, который имеет асимптотику $O(|V| + |E|)$

Задание 3

Предложите $O(|V| + |E|)$ алгоритм поиска максимального по размеру независимого множества в дереве.

Так как нам дано дерево - связный ациклический граф, то здесь независимое множество выглядит примерно так: выберем k -л вершину v_0 в качестве корня этого дерева. Тогда смежными для нее будут являться только ее дети, а их дети смежными являться уже не будут. Аналогично для каждой вершины этого дерева: она является смежной только для своих прямых детей и для своего прямого родителя (существует единственный родитель этой вершины, иначе мы бы получили цикл!). Тогда просто разделив все вершины по четности/нечетности удаления от корня дерево мы получим два независимых множества в этом дереве.

Но мы ищем максимальное, поэтому просто заметим, что для любой вершины верно, что в максимальное независимое множество может входить либо сама эта вершина, либо ее потомки: вместе не могут. То есть на этапе рассмотрения вершины нам необходимо уметь обрабатывать 2 случая: вершина v входит в множество $\Rightarrow$ ее дети не могут входить в множество; вершина v не входит в множество $\Rightarrow$ ее дети могут в него входить. Также важно, что любая вершина дерева образует собственное поддерево, для которого верны все те же рассуждения, что и для целого дерева.

Обычно подобные задачи решаются динпрогом: делим наше дерево на поддеревья, для каждого из них решаем свою задачу, и комбинируем решения. Также по асимптотике я предполагаю, что нужен какой-то обход дерева

Напишем рекурсивный алгоритм:

```
def find_max_ind_set(curr_v):
```

```
    sum_when_isin = 1          # если curr_v входит в множество, то все ее дети точно не входят
    all_v_when_isin = [curr_v] # список вершин, составляющих мн-во
    sum_when_absent = 0        # если curr_v не входит в множество, то дети могут как входить, так и не
    all_v_when_absent = []     # список вершин, составляющих мн-во
```

```
    for v' in curr_v_children:      # идем рекурсивно по всем детям дерева
        # находим максимальные размеры независимых множеств для поддеревьев с корнем в v'
```

```

child_isin, child_absent, vs_child_isin, vs_child_absent = find_max_ind_set(v')

# теперь к sum_when_isin добавляем child_absent - так как вершина
# curr_v не может входить в множество вместе с детьми
sum_when_isin += child_absent
all_v_when_isin += vs_child_absent

# к sum_when_absent добавляем максимум из child_isin, child_absent,
# так как все потомки независимы друг от друга (граф ациклический)
if child_isin > child_absent:
    sum_when_absent += child_isin
    all_v_when_absent += vs_child_isin
else:
    sum_when_absent += child_absent
    all_v_when_absent += vs_child_absent

# функция возвращает максимальное по размеру независимое множество # (максимум из включающих и нет)

return sum_when_isin, sum_when_absent, all_v_when_isin, all_v_when_absent

root = tree.root # выбираем корень дерева
root_isin, root_absent, vs_root_isin, vs_root_absent = find_max_ind_set(root) # запускаем из корня

if root_isin > root_absent:
    print(root_isin, vs_root_isin) # выводим размер и входящие вершины
else:
    print(root_absent, vs_root_absent) # выводим размер и входящие вершины

```

За сколько работает алгоритм? за $O(|E| + |V|)$, так как мы каждую вершину рассматриваем только один раз (то есть алгоритм запускается для нее единственный раз). Само тело алгоритма состоит из нескольких мат операций (сравнение и сложение), работающих за $O(1)$, и мерджа списков вершин, которые не пересекаются (идем снизу вверх, у любой вершины существует единственный родитель - иначе цикличность).

Задание 4

На вход задачи поступает описание двудольного графа $G(L, R, E)$, степень каждой вершины которого равна двум. Необходимо найти максимальное паросочетание в G (которое содержит максимальное количество рёбер). Предложите алгоритм, решающий задачу за $O(|V| + |E|)$.

Граф двудольный $\Rightarrow$ либо ациклический, либо имеет только циклы четной длины (доказала выше).

Рассмотрим просто цикл четной длины. Тривиально, что здесь максимальное паросочетание (их на самом деле два) имеет длину $n/2$, где n - длина цикла, и для его составления нужно просто взять каждое второе ребро.

Здесь не очень понятно как использовать информацию о степенях вершин. Если граф неориентированный, то рассмотрим k -л цикл из n ребер. Каждое ребро соединяет ровно 2 вершины графа, то есть добавляет $+1$ в степень каждой из них. При этом цикл подразумевает, что каждая вершина соединена с двумя другими. Тогда вывод: если в таком графе есть цикл, то он обязательно нечетный, а из условия на степени вершин вытекает, что все циклы изолированы - циклический граф несвязный, каждый цикл изолирован. Значит, мы с помощью DFS находим все циклы, из них выбираем каждое второе ребро. Если общая длина циклов $= n$ (четная, так как сумма четных чисел), то мы получим $n/2$ ребер.

Могут ли быть ациклические подграфы? Ответ - нет, так как любая вершина связана с двумя другими вершинами, а значит. Вообще граф, где степени всех вершин одинаковы, называются регулярными графами, и известно, что 2-регулярный граф состоит из разрозненных циклов. А для каждого цикла алгоритм мы уже нашли.

Тогда алгоритм прост: так как граф состоит из разрозненных циклов четной длины, то нам необходимо лишь сделать обычный обход в глубину DFS, а начинать, например, каждый раз с вершин из L . Считаем

длину цикла n , а к размеру максимального паросочетания сохраняем $n/2$. Алгоритм займет $O(|E| + |V|)$, как и обход DFS.

Задание 5

На вход задачи поступает граф $G(V, E)$ в виде списка исходящих ребер для каждой вершины. Опишите алгоритм, переводящий это представление графа в представление в виде матрицы смежности...

```
GIVEN: # dictionary: v - merged
{a: [b, c, e]
b: [a, k, l]
....
}
```

```
WANTED:
|__|_a_|_b_| ...
|_a_| - | 1 | ...
|_b_| 1 | - | ...
...
```

ALGORITHM

```
all_v = {}
i = 0

for v, merged in merged_dict.items():      # for every mergence list
    if v not in all_v.keys():               # add all v in dict, assign unique index value
        all_v[v] = i                      # 0(1)
        i += 1                            # 0(1)
    for v' in merged:
        if v' not in all_v.keys():
            all_v[v'] = i                  # 0(1)
            i += 1                         # 0(1)

# current total: 0(|E|)
# now we know total number of V and index for every v
# we can make matrix

n = len(all_v.keys())                      # |V|
matrix = [[0]*n]*n                         # |V|*|V| matrix

for v, merged in merged_dict.items():      # for every mergence list
    v_index = all_v[v]                    # v index in matrix indexes

    for v' in merged:
        v'_index = all_v[v']              # v' index in matrix indexes
        matrix[v_index, v'_index] = 1     # we have an edge here

TOTAL: 0(|E|)
```

Самым трудозатратным этапом будет сборка всех вершин: нам нужно найти все вершины, которые есть в данном графе. Замечу, что на граф не накладывается никаких условий, поэтому он может быть как орграфом, так и неорграфом: мы не можем взять ключи из словаря со списками смежности, так как может существовать вершина орграфа, в которую ребра только заходят, но не выходят из нее. Поэтому нам придется пройти по каждому списку словаря смежности: мы это сделаем за $O(|E|)$: так как количество

ключей оценивается через $|V|$, а $len(arr) = |e(v, u)|$, где v - вершина из ключа, то общая длина списков равна $|E|$.

Эти вершины мы будем класть в новый словарь вида $v : i$, где v - вершина, а i - ее порядковый номер. Добавление и извлечение из хеш-таблиц выполняется за $O(1)$. Эта операция нужна для того, чтобы конвертировать названия вершин в индексы нашей таблицы, от 0 до $|V|-1$.

Теперь `all_v` содержит все вершины, из которых исходят ребра, а также их индексы в нашей матрице смежности.

Создадим матрицу смежности. Пусть это выполняется за $O(1)$, например, списком списков в питоне.

Теперь мы еще раз пройдемся по всем спискам смежности. Теперь мы можем по индексам вершин из `all_v` добавить каждое ребро за $O(1)$ путем доступа по индексу, тогда всего $O(|E|)$ операций.

Таким образом:

- создать индекс для каждой вершины, из которой выходит ребро, можно за $O(|E|)$
 - создать пустую матрицу смежности - условно $O(1)$
 - заполнить матрицу смежности - за $O(|E|)$
- total: $O(|E|)$

Задание 6

...и наоборот. Докажите корректности и оцените сложность обоих алгоритмов.

Когда у нас дана матрица смежности, то она имеет размер $|V|*|V|$. При этом у нас названия вершин - если нужны конкретные, например, буквенные, могут быть заданы индексами самой матрицы, или лежат в отдельном словаре. Тут допустим, что операция доступа к имени вершины у нас осуществляется за $O(1)$ путем индексирования или доступа к словарю индекс-название.

GIVEN:

```
|_|_|_|_a_|_b_| ...  
|_a_| - | 1 | ...  
|_b_| 1 | - | ...
```

WANTED:

```
...  
{a: [b, c, e]  
b: [a, k, l]  
....  
}
```

ALGORITHM

```
mergence_dict = {}  
n = len(matrix)  
  
for i in range(n):  
    v = names[i] # |V| times  
    v_list = [] # get v name, O(1)  
    for j in range(n):  
        if matrix[i,j] == 1: # |V| times  
            v_list.append(names[j]) # O(1)  
    mergence_dict[v] = v_list # append v', O(1)
```

TOTAL: $O(|V|^2)$

Итого мы просматриваем каждую ячейку матрицы смежности и обрабатываем ее за $O(1)$. Так как размер матрицы $|V|*|V|$, то сложность $O(|V|^2)$